ZikShare

ACADEMIC VAULT

UNIZIK FRESHER EDITION | 2026

CSC 101: Introduction to Computer Science - Hardware, Data and Algorithms

What actually happens inside a computer, and how to think in algorithms

FACULTY PRIMER

Faculty Primer

Faculty of Physical Sciences
Computer Science | 100L

Written for students, by students.

Practical notes, worked examples, practice questions and a revision plan.

Free to download. No fee, no payment step, no watermark.

Independent study support | Not an official UNIZIK publication

START HERE

Why this pack exists

CSC 101 is often taught as a tour of parts, but the exam tests understanding. This pack explains the stored-program model, number systems, data representation, operating systems, networks and algorithmic thinking, with worked binary and flowchart exercises.

This is a student-written study companion, not a reproduced lecture handout or a leaked examination paper. Use it to organise the topics you are taught, practise the kind of thinking your course rewards, and walk into class with better questions.

What you will be able to do

- Explain the stored-program architecture and the roles of CPU, memory and storage.
- Convert between binary, decimal and hexadecimal and perform simple binary arithmetic.
- Represent text, images and sound in digital form and explain the trade-offs.
- Design a simple algorithm as pseudocode and a flowchart, and trace it for correctness.

How to use this pack

1. Skim the whole pack once in 20 minutes so you know what is inside.
2. Work through one section at a time with your lecture note and recommended text beside you.
3. Attempt the practice questions before reading the answers. Mark what you missed.
4. Use the final checklist to plan your revision week, one tick per study session.

> **Free to download, always**
> This pack is free. There is no fee, no subscription and no payment step before download, and the full PDF is not watermarked or locked. Share the link with a coursemate instead of forwarding the file.

> **Confirm your official course outline**
> Course codes, credit units and outlines can differ by department and session. Treat this pack as supplementary support and confirm the current outline with your lecturer, course rep or departmental handbook before an assessment.

The stored-program model

Modern computers follow the idea that instructions and data live in the same memory and are fetched and executed in sequence.

- CPU: control unit fetches and decodes, arithmetic logic unit executes.
- Registers: small, fast storage inside the CPU.
- Main memory: volatile, holds running programs and data.
- Secondary storage: persistent, slower, larger.
- Input and output devices move data between the machine and the world.

- The fetch-decode-execute cycle repeats for each instruction.

Number systems and data representation

Computers store patterns of bits. Everything else is a convention layered on top.

- Binary to decimal: sum each bit times its place value.
- Decimal to binary: divide repeatedly by 2 and read remainders upward.
- Hexadecimal groups four bits and is used for compact memory addresses and colours.
- One byte is eight bits; ASCII uses seven or eight bits per character, Unicode uses variable encoding.
- Images are stored as pixel values; resolution and colour depth trade quality against size.
- Sound is sampled at intervals; sampling rate and bit depth determine fidelity and file size.

> **Practice conversion by hand**
> Exams rarely allow calculators for number-system questions. Convert 156 to binary and back until it takes under a minute.

Algorithms, pseudocode and flowcharts

An algorithm is a finite, unambiguous sequence of steps that solves a problem.

- Sequence, selection and iteration are the three control structures.
- Pseudocode should be readable by a human and precise enough to implement.
- Flowchart symbols: oval for start and end, parallelogram for input/output, rectangle for process, diamond for decision.
- Trace a loop by writing a table of variable values for each iteration.
- Test with normal, boundary and invalid inputs.
- An algorithm that produces an answer but never terminates is not correct.

What a computer actually does

A computer is a machine that executes instructions. Its architecture comprises a processor, memory, input and output devices, and storage, connected by buses. The processor fetches an instruction, decodes it, executes it and writes the result back - the fetch-decode-execute cycle, repeated billions of times per second.

Programs and data share memory in the stored-program model, which is what makes general-purpose computing possible: the machine does not change, the instructions do. Memory hierarchy - registers, cache, RAM, disk - trades speed against capacity and cost, and understanding that hierarchy explains most performance behaviour.

Data is represented in binary because digital circuits distinguish two voltage levels reliably. Numbers, characters and images are all encoded: two's complement for signed integers, ASCII and Unicode for characters, pixel values for images. Knowing the encoding explains overflow, rounding error and why text files have encodings at all.